OriginTrace Security Console

Runtime to Source DevSecOps Operator Guide



Runtime signals flow through AST correlation into verified source controls

This guide explains how to investigate a kernel-level security event, locate the responsible code, verify exploit reachability, generate a focused Semgrep rule, and stage a safer implementation for review.

Repository owner gocar112
Edition 2026
Live console origintrace-security-console.starage100.chatgpt.site

Contents

1 Why runtime and static evidence must meet

2 System architecture

3 Operating the security console

4 OWASP surface scanning

5 Runtime event ingestion

6 AST source correlation

7 Dual verdict reachability

8 Semgrep rule synthesis

9 Remediation and pull request staging

10 Elastic and Kibana observability

11 MCP and agent workflows

12 Deployment and operational safeguards

A Incident field reference

B Source repositories

1. Why runtime and static evidence must meet

Static analysis finds possible defects, while runtime detection proves that a process performed a sensitive action. OriginTrace combines these signals so teams can distinguish reachable production risk from code that has never executed.

The context gap

SAST findings often identify dangerous functions without knowing whether a route, job, or handler can reach them. Falco and other eBPF tools detect process and file activity inside containers, but a syscall alone does not name the source file that caused it. The gap between those systems creates duplicate triage and alert fatigue.

The combined decision

- Use OWASP surface probes to identify externally observable weaknesses and response indicators.
- Use Falco runtime events to establish that a sensitive syscall occurred inside a workload.
- Use Python AST correlation to resolve the syscall to a dangerous sink in a file and function.
- Use Semgrep taint analysis to verify whether untrusted input can reach that sink.
- Stage a focused patch for developer review instead of silently changing production code.

2. System architecture

The platform preserves evidence as it moves from detection to remediation. Each stage adds context without discarding the original event.

Stage	Component	Purpose	Primary output
1	Falco webhook	Capture anomalous kernel activity	Normalized runtime event
2	FastAPI receiver	Validate and persist telemetry	Incident JSON
3	Correlation engine	Map syscall to Python AST sink	File, line, function
4	Reachability engine	Combine runtime and static proof	Score and verdict
5	Rule synthesizer	Generate a focused taint rule	Semgrep YAML
6	Remediation stage	Replace unsafe behavior	Unified diff
7	Elastic pipeline	Retain and visualize evidence	Forensic audit stream

Evidence continuity

The incident identifier, command line, workload metadata, source origin, rule identifier, and remediation reference should remain linked. This allows an analyst to move from a dashboard row to the exact code change without reconstructing the event from several tools.

3. Operating the security console

The console opens on the incident queue because investigation is the primary task. The summary metrics provide context, but the selected incident carries the evidence needed for action.

1. Filter the queue to confirmed or static-only findings, or search by syscall, rule, source path, OWASP category, or CWE.
2. Select an incident and verify the severity, reachability score, runtime command, source origin, and function.
3. Compare the vulnerable sink with the staged remediation before opening the generated Semgrep rule.
4. Use the reachability lab to test how each evidence gate changes priority.
5. Move to the OWASP scanner view when an authorized surface assessment is required.

Repository evidence mode

The hosted console contains scenarios documented in the source repositories. It does not claim that a live cluster is connected. When a backend is added, the interface should label live, replayed, and repository evidence explicitly.

4. OWASP surface scanning

The companion Python scanner performs rate-limited HTTP checks for common OWASP Top 10 indicators. The hosted console prepares a command; the trusted local environment executes the network test.

Check family	Example signal	Typical category	Priority
SQL injection	Database error after crafted parameter	A03 Injection	High
Reflected XSS	Payload returned in an executable context	A03 Injection	Medium
Command injection	Shell output after command delimiter	A03 Injection	High
File inclusion	Sensitive file content in response	A01 Access control	High
Security headers	Server or framework details exposed	A05 Misconfiguration	Low
Component versions	Version string exposed in content	A06 Components	Low
Log exposure	Trace or exception text in response	A09 Logging	Low

Run an authorized scan

```
python ScannerWebSite.py --url "https://app.example.com" --output scan-report.txt
```

Only assess systems you own or have explicit permission to test. Keep the scanner delay enabled, preserve TLS verification, and review requests before testing production endpoints.

5. Runtime event ingestion

Falco sends a structured event when a monitored workload performs suspicious process, file, or network activity. The receiver validates the event and stores a normalized incident record.

Essential event fields

Field	Meaning	Investigation use
rule	Falco detection rule	Explains why the event fired
priority	Falco severity	Provides an initial triage signal
proc.cmdline	Observed command line	Preserves runtime intent and parameters
proc.pid	Process identifier	Links adjacent process evidence
container.name	Container identity	Identifies the execution boundary
k8s.pod.name	Kubernetes pod	Links the event to a workload instance
time	UTC event time	Supports timeline reconstruction

Persistence

OriginTrace writes normalized incidents to disk using timestamped JSON files. This supports offline triage and provides a simple source for the CLI and MCP server. Elastic ingestion can run in parallel for search and dashboard use.

6. AST source correlation

The correlation engine walks Python syntax trees and searches for sinks that can explain the observed syscall. The result is an exact file, line, function, and call expression rather than a repository-wide guess.

Runtime syscall	Representative Python sinks	Risk class
execve	os.system, subprocess.Popen, subprocess.run	Command execution
openat	open, Path.read_text, Path.open	File and secret access
connect	socket.connect, requests.get, subprocess network tools	Outbound connection or SSRF

Correlation constraints

- A sink match is a candidate until runtime metadata and data flow support it.
- Generated or vendored code should be separated from first-party application paths.
- Line numbers must come from the analyzed revision, not from an unrelated checkout.
- Several candidate sinks may require route, process, or command-signature disambiguation.

7. Dual verdict reachability

Reachability scoring prevents all findings from competing at the same priority. The score records which forms of evidence are present and produces a verdict that analysts can audit.

Evidence gate	Weight	What it proves
Runtime observed	35	A sensitive action occurred in a monitored workload
AST sink matched	25	A source call can explain the syscall
Semgrep verified	25	A taint path reaches the sink
External input reachable	15	An attacker-controlled entry point reaches the path

Verdict guidance

A high score should not hide its inputs. The console exposes each gate so an analyst can see why an incident is confirmed, under investigation, or static only. A confirmed verdict requires runtime evidence and a matching source sink, not merely a numeric threshold.

8. Semgrep rule synthesis

For a confirmed path, the synthesizer creates a taint rule that matches the observed sink and likely request sources. The rule records CWE, OWASP, and incident metadata so the control remains explainable in CI.

```
rules:
  - id: auto-origintrace-terminal-shell
    mode: taint
    severity: ERROR
    pattern-sources:
      - pattern: request.query_params.get(...)
    pattern-sinks:
      - pattern: subprocess.run(...)
    metadata:
      cwe: CWE-78
      owasp: A03:2021 - Injection
```

Verification before adoption

1. Validate the generated YAML.
2. Run the official Semgrep CLI against the affected repository revision.
3. Confirm the rule finds the intended sink without flooding unrelated code.
4. Add a positive and negative test case before enforcing the rule in CI.

9. Remediation and pull request staging

OriginTrace stages changes for review. A patch should remove the dangerous behavior, validate external input, preserve required functionality, and remain narrow enough for a developer to understand.

Command execution example

```
- os.system(f"ping -c 1 {host}")
+ validate_host(host)
+ subprocess.run(["ping", "-c", "1", host], check=True, shell=False)
```

Review checklist

- Confirm the fix addresses the observed runtime path and not only a similar static pattern.
- Keep shell execution disabled when an argument-vector API is available.
- Validate allowlists, path boundaries, timeouts, and failure behavior.
- Run application tests and the synthesized Semgrep rule against the patched revision.
- Require human review before merge and preserve the incident reference in the change record.

10. Elastic and Kibana observability

The optional Elastic pipeline makes reachability and code-origin fields searchable alongside the original Falco telemetry. Dashboards should help analysts prioritize and reconstruct events, not replace the underlying evidence.

Visualization	Question answered	Key fields
Reachability donut	How much of the queue is confirmed?	verdict, score
Severity histogram	Which priorities dominate?	falco.priority
Audit stream	What happened and where?	time, command, file, line
Source hot spots	Which code locations recur?	file, function, sink

Retention considerations

Runtime commands and file paths can contain sensitive operational data. Limit access, define retention, avoid ingesting secrets, and document whether an event came from production, a replay, or a test workload.

11. MCP and agent workflows

The JSON-RPC MCP server exposes focused tools for incident discovery, correlation, and rule synthesis. The tool boundary helps an AI coding agent request evidence without unrestricted access to the entire security pipeline.

Tool	Purpose	Expected result
query_runtime_incidents	List incidents by priority	Incident identifiers and commands
correlate_incident_to_code	Resolve an incident to source	File, line, function, sink
synthesize_semgrep_rule	Create a focused prevention rule	YAML rule and metadata

Safe agent use

- Require explicit authorization before scanning external targets or changing repositories.
- Keep remediation patches staged until a human reviews them.
- Return evidence and uncertainty with every verdict.
- Do not expose secrets, full token files, or unnecessary runtime payloads to downstream tools.

12. Deployment and operational safeguards

The console is a static interface that can be deployed independently from the Python correlation services. This separation makes the evidence boundary explicit and prevents the browser from claiming capabilities it does not have.

Deployment checklist

1. Validate JavaScript syntax and all local asset references.
2. Confirm repository evidence is labeled and no production credentials are present.
3. Preview desktop and mobile layouts before publishing.
4. Package only the configured static directory and hosting manifest.
5. Verify the deployment status and retain the returned production URL.
6. Keep the GitHub repository and AIOS project copy synchronized with the published source.

Desktop access

The included AIOS skill can create a Windows desktop shortcut with the OriginTrace icon. The shortcut opens the private live Site in the user's default browser.

A. Incident field reference

Use these fields as the minimum contract between runtime detection, correlation, the console, and observability systems.

Field	Type	Description
incident_id	string	Stable identifier across the evidence chain
timestamp	UTC datetime	Time of the runtime event
priority	enum	Falco severity such as CRITICAL or WARNING
rule	string	Runtime rule that generated the event
command_line	string	Observed process command
syscall	enum	execve, openat, connect, or another mapped syscall
source_file	path	Repository-relative source location
source_line	integer	Line number for the matched sink
function	string	Containing function or route handler
sink	string	Dangerous call matched by AST analysis
reachability_score	0 to 100	Weighted evidence score
verdict	enum	CONFIRMED_EXPLOITABLE or STATIC_ONLY
semgrep_rule	path	Generated rule location
patch	path	Staged unified diff

B. Source repositories

The console combines concepts and workflows from two repositories and keeps their responsibilities distinct.

OriginTrace DevSecOps

https://github.com/ritvikindupuri/OriginTrace_DevSecOps

Provides the runtime webhook receiver, incident store, Python AST correlation, reachability logic, Semgrep rule synthesis, patch staging, MCP server, and Elastic integration.

OWASP Top 10 Vulnerability Scanner

https://github.com/gocar112/the-OWASP-Top-10-Vulnerability-Scanner

Provides rate-limited HTTP checks for injection, authentication, file inclusion, misconfiguration, component disclosure, and logging indicators.

OriginTrace Security Console

https://origintrace-security-console.starage100.chatgpt.site

Presents repository-backed incidents, source correlation, reachability evidence, staged remediation, generated controls, and authorized scanner command preparation in one responsive interface.